

DAE-VAE

Deep learning project. A DAE & a VAE that denoises and generates images of apples or bananas. Also made by Carlos Moreno.

Juan Pablo Colomé

Created on March 26 | Last edited on March 31

PREPROCESS

- Contrast Limited Adaptive Histogram Equalization (CLAHE)



¿Qué hace?

Es una técnica para mejorar el contraste de una imagen, pero de forma local y controlada.



Diferencias frente a la ecualización normal:

Histograma global: Mejora el contraste de toda la imagen como un todo.

CLAHE (adaptive): Divide la imagen en bloques y mejora el contraste por bloque.

Contrast limited: Limita cuánto se amplifican los valores para evitar que aumente el ruido.



¿Por qué se usa CLAHE aquí?

Las imágenes de frutas pueden tener sombras, fondos opacos o iluminación desigual.

CLAHE ayuda a:

Realzar detalles como bordes o texturas de la fruta.

Hacer más uniforme la distribución de los colores.

Mejorar el aprendizaje de los modelos, sobre todo en entornos no controlados.

2. Redimensionar a 224 x 224

 ¿Qué significa?

Todas las imágenes se transforman a tamaño 224 píxeles de alto × 224 píxeles de ancho, con 3 canales (RGB).

(224, 224, 3)

 ¿Por qué ese tamaño?

Es el mencionado por Google como el tamaño ideal.

Una idea de como funcionan:

Tamaño	Pros	Contras
32x32	Muy rápido de entrenar	Pierdes muchos detalles visuales
128x128	Buen nivel de detalle	Aún eficiente en entrenamiento
256x256+	Mucha más información visual	Entrenamiento más lento y pesado

Usar 224x224 es ideal para:

No perder la forma ni colores de las frutas.

Entrenar en Google Colab sin necesidad de GPU muy potente.

Mantener el dataset en un tamaño manejable.

3. Normalización a [0, 1]

Al dividir todos los valores de píxeles entre 255:

```
img_array = img_array / 255.0
```

Escalas los datos a un rango estándar para que la red neuronal converja mejor.

Es más fácil aprender cuando los valores están entre 0 y 1 que si están entre 0 y 255.

4. Split de datos (Entrenamiento, Validación, Prueba)

El conjunto se divide así:

Entrenamiento: 70%

Validación: 15%

Prueba: 15%

Esto permite:

Entrenar el modelo.

Validar durante el entrenamiento y ajustar hiperparámetros.

Probar al final para saber qué tan bien generaliza.

Además, se uso stratify=y para que la cantidad de imágenes de manzanas y bananas se mantenga balanceada en los tres conjuntos.

DAE

🔧 ¿Qué tipos de ruido usaste?

1. Ruido Gaussiano

Añade valores aleatorios siguiendo una distribución normal.

Simula "interferencia" o sensores con mala calidad.

```
noise = np.random.normal(0, intensidad, img.shape)
```

2. Ruido Sal y Pimienta

Píxeles aleatorios se vuelven blancos o negros.

Simula fallas en sensores o corrupción de datos.

```
rnd = np.random.rand(*img.shape)
```

```
noisy[rnd < prob / 2] = 0 # negro
```

```
noisy[rnd > 1 - prob / 2] = 1 # blanco
```

🧱 ¿Cómo es el DAE?

🔧 Arquitectura:

ENCODER

Capa	Salida esperada	Detalles
Conv2D(32, 3x3)	(112, 112, 32)	Conv. con 32 filtros, kernel 3x3, stride 2, padding "same", activación ReLU
Conv2D(64, 3x3)	(56, 56, 64)	Conv. con 64 filtros, mismo setup
Conv2D(128, 3x3)	(28, 28, 128)	Más filtros para captar más abstracción
GlobalAveragePooling2D	(128,)	Promedia cada uno de los 128 mapas de activación
Dense(latent_dim=64)	(64,)	Vector latente final (codificación compacta)

DECODER

Entrada: (64,) → vector latente

Capa	Salida esperada	Detalles
Dense(28×28×64)	(50176,)	Reconstruye estructura espacial desde el vector latente
Reshape	(28, 28, 64)	Da forma de "imagen intermedia"
Conv2DTranspose(64, 3×3)	(56, 56, 64)	Upsampling usando transposed convs
Conv2DTranspose(32, 3×3)	(112, 112, 32)	Más detalle progresivo
Conv2DTranspose(16, 3×3)	(224, 224, 16)	Último upsampling para recuperar tamaño original
Conv2D(3, 3×3)	(224, 224, 3)	Imagen final reconstruida, activación <code>sigmoid</code> (valores en [0, 1])

⚙️ Hiperparámetros:

Función de pérdida: `binary_crossentropy`

Optimizador: Adam

Épocas: Hasta 30 con early stopping

Batch size: 32

(

- Suficientes muestras para estimar bien el gradiente.
- Sin exigir demasiado a la memoria de la GPU o CPU.
- Te permite entrenar con un buen ritmo sin desestabilizar el aprendizaje

)

📉 ¿Por qué `binary_crossentropy`?

Aunque las imágenes son RGB, al usar activación `sigmoid`, la salida es un valor entre [0, 1] por píxel, lo que se interpreta como una

probabilidad de que el píxel esté activado.

¿Por qué no mse?

MSE asume distribuciones gaussianas (regresión). BCE funciona mejor en salidas tipo "on/off" por píxel.

1. Métrica de pérdida (binary_crossentropy):

Si empieza alto (ej. > 0.2) y baja cerca de 0.01 o menos, significa que el modelo está aprendiendo bien a reconstruir.

Lo comparas con el conjunto de validación, y si la pérdida valida también baja, no está sobreajustando.

O sea, si inicia en un punto y acaba en un punto más bajo es porque esta aprendiendo pero si en el conjunto de validación no esta pasando lo mismo, es porque se esta sobreajustando.



Run set 30





VAE

 Arquitectura del VAE en el notebook

 Encoder

Nº	Tipo de Capa	Descripción	Tamaño de salida
1	Input	Entrada de imagen (224, 224, 3)	224×224×3
2	Conv2D	32 filtros, stride 2, padding same	112×112×32
3	Conv2D	64 filtros, stride 2	56×56×64
4	Conv2D	128 filtros, stride 2	28×28×128
5	Conv2D	256 filtros, stride 2	14×14×256
6	Flatten	Aplana a vector 1D	50176
7	Dense	512 unidades, activación ReLU	512
8	Dense (×2)	<code>z_mean</code> y <code>z_log_var</code> → tamaño 256 c/u	(256, 256)
9	Sampling (custom)	Muestra vector <code>z</code> ∈ $\mathbb{R}^{256}$	256

✓ Total: 9 capas lógicas (8+1 Sampling) ↓

◆ Decoder

Nº	Tipo de Capa	Descripción	Tamaño de salida
1	Input	Vector latente <code>z</code> de tamaño 256	256
2	Dense	14×14×256 unidades (igual que fin del encoder)	50176
3	Reshape	Acomoda a forma 3D	14×14×256
4	Conv2DTranspose	256 filtros, stride 2	28×28×256
5	Conv2DTranspose	128 filtros, stride 2	56×56×128
6	Conv2DTranspose	64 filtros, stride 2	112×112×64
7	Conv2DTranspose	32 filtros, stride 2	224×224×32
8	Conv2DTranspose	3 filtros, activación sigmoid	224×224×3

✓ Total: 8 capas ↓

🔧 Clase VAE personalizada

Esta clase define cómo se entrena el modelo. En cada paso de entrenamiento:

Codifica la imagen → obtiene `z_mean`, `z_log_var`, `z`.

Reconstruye la imagen usando el decoder.

Calcula dos pérdidas:

Reconstrucción: qué tan similar es la imagen reconstruida a la original (usa binary crossentropy por ser imágenes normalizadas)

entre 0 y 1).

KL divergence: qué tanto difiere la distribución aprendida de una gaussiana estándar (para regular el espacio latente).

Calcula la pérdida total: reconstrucción + KL.

 ¿Qué es el FID?

El Fréchet Inception Distance (FID) es una métrica de calidad para imágenes generadas. Compara dos distribuciones:

Una de imágenes reales.

Una de imágenes generadas.

Se usa un modelo InceptionV3 para extraer características (features) de ambas imágenes. Luego se comparan sus estadísticas (media y covarianza).

Interpretación:

FID bajo = imágenes generadas son similares a las reales.

FID cercano a 0 es ideal.

< 50 ya es considerado aceptable.

☒

Run set 30

⋮



Interpretación de las métricas

Métrica Significado

FID: 14.71 No tan bueno. Las imágenes generadas no son muy similares a las reales.

epoch/epoch: 29 El modelo entrenó 29 épocas.

epoch/kl_loss: 83.55 Pérdida KL: qué tan lejos está la distribución latente de una normal estándar. Valores bajos son mejores pero no deben ser 0 (sino pierde variabilidad).

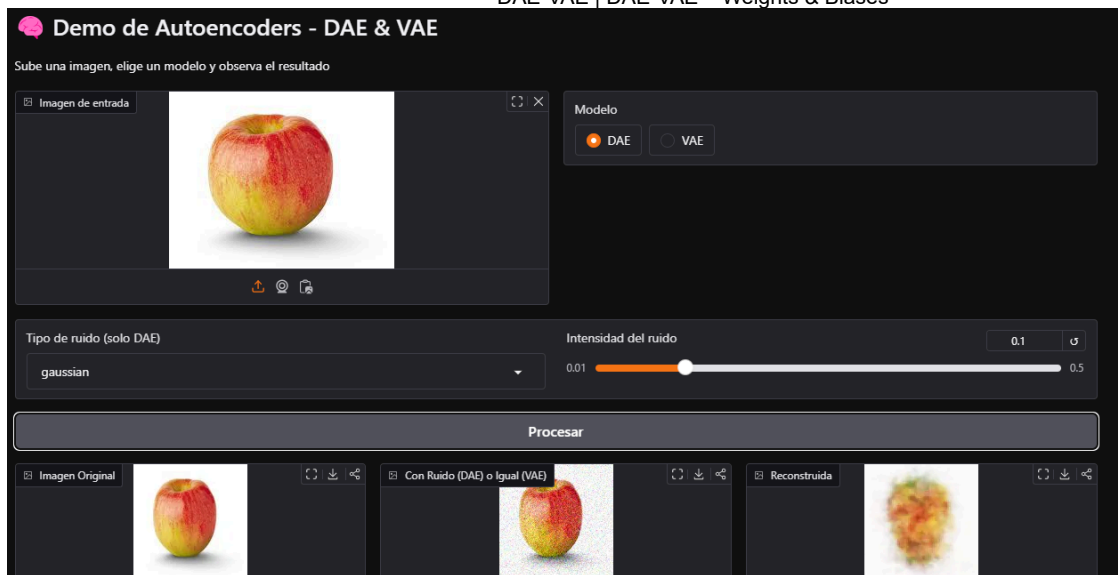
epoch/reconstruction_loss: 8575.83 Pérdida de reconstrucción. Mientras más baja, mejor está reconstruyendo.

epoch/loss: 8659.38 Pérdida total = reconstrucción + KL.

val_* Métricas en el conjunto de validación. Similaridad con las métricas de entrenamiento indica que no hubo overfitting.

DEMO

Guardamos los modelos, creamos una interfaz en gradio y la hosteamos en hugging face para poder hacer uso de los modelos.



CONCLUSIÓN

Fue un proyecto bastante retador. Hubo problemas desde el inicio, y más que nada en el inicio, hasta el final. El web scraping fue muy difícil y concluimos que las imágenes del dae y vae no fueron tan buenas por que hubo poca cantidad de datos. Los modelos después de intentar diferentes redes, con diferentes capas y con diferentes hiperparámetros, no mostraron mejoras y por eso llegamos a esa conclusión. Descargar los modelos para usarlos en el demo también nos causo muchos problemas por compatibilidad de librerías y dependencias, pero al final se logró.

Created with ❤️ on Weights & Biases.

<https://wandb.ai/juan-colome-iteso/DAE-VAE/reports/DAE-VAE---VmldzoxMTk5NjM4Nw>